3D SED Classical Hydrogen Simulator

This repository contains the C++ computational physics engine and Python post-processing suite used to simulate the classical hydrogen atom in Stochastic Electrodynamics (SED).

The software utilizes a pseudo-wave formulation in k-space, logarithmic frequency gridding, and a Tukey smoothing window to resolve historical computational bottlenecks regarding memory footprint, execution speed, and numerical discontinuities. Trajectory integration is handled via a robust adaptive Runge-Kutta fourth-order (RK4) algorithm with step-doubling (Richardson Extrapolation).

The article that discusses much of the following is entitled, “Advanced computational methods for simulating the classical hydrogen atom in stochastic electrodynamics,” by Daniel Cole.

Repository Structure

- main.cpp: The core 3D C++ simulation engine.

- config.json: The control file for simulation variables.
- json.hpp: The Niels Lohmann JSON library required for parsing the configuration.
- NetHistogram-3-curves.py: Python post-processing script for generating time-weighted probability density histograms using Kernel Density Estimation (KDE).

Requirements & Compilation

C++ Engine

The code requires a compiler supporting C++11 or higher (e.g., GCC, Clang, or Apple LLVM via Xcode). For maximum performance, compile with aggressive optimization and fast-math flags.

Bash

```
g++ -O3 -ffast-math main.cpp -o sed_sim
```

Python Post-Processing

The visualization scripts require Python 3 and standard scientific libraries:

Bash

```
pip install numpy scipy matplotlib
```

Execution

Ensure config.json is located in the same directory as the compiled executable. You can run the simulation with optional batch prefixes and run numbers (useful for generating multi-run ensembles):

Bash

./sed_sim [BatchPrefix] [RunNumber]
Example: ./sed_sim Batch-5.3A 001 will produce Batch-5.3A-Trajectory-001.bin and Batch-5.3A-Histogram-001.csv.

Configuration Parameters (config.json)

The engine is controlled entirely through config.json without needing to recompile the source code.

simulation_parameters

- rng_seed: Base seed for the pseudo-random number generator (automatically offset by the run number for ensembles).
- toggles: Enable or disable specific physical forces

(e.g., enable_coulomb_rr, enable_magnetic_lorentz, enable_zp).

initial_conditions

- a_start: The starting orbital radius of the electron in centimeters (e.g., 0.53×10^{-8} for the Bohr radius).

vacuum_parameters

- num_frequency_bins: Total discrete modes in the logarithmic ZP frequency grid (recommended: 1,000,000 to 100,000,000).

- window_width_percent: The dynamic active sliding window width tracking the Keplerian frequency (e.g., 0.05 for $\pm 5\%$).

- taper_width_percent: The fractional width of the Tukey cosine smoothing function applied to the window edges to prevent numerical discontinuities.

integration_parameters

- t_max: Total physical simulation time in seconds (e.g., 300 ps = 300e-12).

- dt_initial: The starting time step size for the adaptive integrator.

- dt_max / dt_min: The absolute ceiling and floor for the adaptive time step.
- tolerance: The maximum allowed spatial error between a full RK4 step and two half-steps.
- tracker_stride: Down-sampling parameter for ZP field calculations in deep orbits.

applied_fields

- Allows the injection of external classical electromagnetic waves (e.g., linear or circular laser pulses) into the system.

output_configuration

- printInterval: Determines how often the adaptive RK4 steps are written to the binary .bin output file.

Output Files

1. **Binary Trajectory (.bin)**: A continuous recording of the electron's dynamical state. The 8-double data structure per frame is: [Time, Radius, Semi-major Axis, Eccentricity, Theta, Energy, |L|, L_z].
2. **CSV Histogram (.csv)**: A pre-binned radial distribution capturing the electron's position after

passing the initial 50,000-step stabilization period.

Python scripts for generating plots

Please see the article, “Advanced computational methods for simulating the classical hydrogen atom in stochastic electrodynamics.” There are three figures shown in Sect. 6.

Here is how to produce the first figure in Sect. 6.0. This figure has four “panels”, of *r* , *a* (semimajor axis), eccentricity, and polar angle of the normal of the plane of orbit to the *z* axis, all as functions of time, *t*, on the horizontal axis. Download python3 version. Use the terminal window containing the script and trajectory.bin file as in:

python3 plot_trajectory.py name-trajectory-file.bin

Note that you can zoom in on the figures, and using the arrow keys, can return to your first image.

To create the next figure, here are two scripts that can be used. The first one creates one histogram, as in the article. Place all the trajectory-file.bin files that you have created into one folder. Now, some files will have the trajectory up to say, 10.1 ps, another to 15.5 ps, etc. - they will likely have different “electron times”, since the larger orbits are calculated faster in real time than the smaller

orbits, which take much smaller time steps (adaptive step size). Edit the script file and put a time in at:

MAX_TIME_S = 8.0e-12

This will include all trajectory files that have a final time GREATER than 8 ps. Those files less than an ending time of 8 ps will not be included in the histogram that is created.

python3 NetHistogram-rescue-All.py /location/folder-with-traj-files

This will produce the second figure (histogram) in Sect. 6.

An alternative variation of this figure is the following script

python3 NetHistogram-3-curves.py /location/folder-with-traj-files

Edit this file and change the times (throughout the script) to three times, like TIMES_PS = [2.0, 7.5, 31.0] for 2,7.5,31.0 ps.

You will then produce three histograms laid on top of one another.

Finally, for the last figure in Sect. 6, you can produce a figure with three panels, one of energy, second of

magnitude of the angular momentum, and third of the z component of the angular momentum.

python3 plot_energy.py name-trajectory-file.bin

A useful script is bin_to_ascii.py, used as:

python3 bin_to_ascii.py name-of-traj.bin

A *.txt file is produced that can be plotted, for example, in textplot, using any/all of the columns in the *.bin file of:

r,a,ecc,Theta-n,E,L,Lz

To plot the histograms that occur in the csv files, use:

python3 plot_histogram.py histogram-file-name.csv